

Reto. Dada la siguiente clase:

```
#include <cstdlib>
#include <iostream>

using namespace std;

class Punto {
    int x,y;
public:
    Punto() { x = 0; y = 0; }
    Punto(int a) { x = a; y = 0; }
    Punto(int a, int b) { x = a; y = b; }
    Punto add(Punto p) {
        x+=p.x; y+=p.y; return (*this);
    }
    Punto add(Punto p, Punto q) {
        x=x+p.x+q.x;
        y=p.y+q.y;
        return (*this);
    }
    void inc23() { x+=2; y+=3; }
    void inc23(Punto p) { x+=p.x; y+=p.y; }
    void ver() const { cout << x <<" "<< y << endl; }
};

int main() {
    Punto x(3,2), y;
    const Punto z=1; //const Punto z(1); const Punto z=Punto(1);
    cout << "x, y, z valen:\n";
    x.ver(); y.ver(); z.ver();
    y.add(x, z);
    x=y.add(z);
    cout << "x, y, z valen:" << endl;
    x.ver(); y.ver(); z.ver();
    x.inc23(); y.inc23(z);
    cout << "x, y, z valen:" << endl;
    x.ver(); y.ver(); z.ver();

    //haz que sea posible ejecutar las lineas comentadas sin cambiar nada en la clase Punto
    //excepto añadir un método a la clase Punto y una funcion amiga
    /*
    x = y + z; cout << "x: "; x.ver();
    x = z + x; cout << "x: "; x.ver();
    y = z + 4; cout << "y: "; y.ver();
    y = 2 + z + y +1 ; cout << "y: "; y.ver();
    cout << "x: " << x << " y: " << y << " z: " << z << endl;
    */
    system("PAUSE"); return EXIT_SUCCESS;
}
```

SALIDA POR PANTALLA

```
x, y, z valen:
3,2
0,0
1,0
x, y, z valen:
5,2
5,2
1,0
x, y, z valen:
7,5
6,2
1,0
Presione una tecla para continuar . . .
```

Haz que sea posible ejecutar las líneas comentadas sin cambiar absolutamente nada en la clase Punto excepto añadir un nuevo método y una función amiga a la clase Punto.

SOLUCION:

Tenemos que sobrecargar el operador+ para poder sumar *Puntos con Puntos*, *Puntos con enteros* y *enteros con Puntos*. Los 2 primeros podemos hacerlo con métodos (es lo recomendado) y el último (*entero con Puntos*) solo podemos hacerlo mediante una función.

También tenemos que sobrecargar el *operator<<* para poder usarlo en el `cout <<` (el *operator<<* sólo se puede sobrecargar mediante funciones, no se puede sobrecargar con métodos).

Como en el enunciado nos dicen que sólo podemos añadir un método y una función amiga a la clase optamos por:

- sobrecargar la *suma de Puntos con Puntos* con un método
- sobrecargar la *suma de Puntos con enteros* y *de enteros con Puntos* con funciones (ajenas a la clase)
- sobrecargar el *operator<<* con una función amiga (es la única forma de poder implementarla)

El método suma de Puntos con Puntos puede ser invocado por un objeto Punto constante por lo que habrá que etiquetar dicho método como constante para que pueda ser utilizado por dicho objeto.

```

#include <cstdlib>
#include <iostream>

using namespace std;

class Punto {
    int x,y;
public:
    Punto() { x = 0; y = 0; }
    Punto(int a) { x = a; y = 0; }
    Punto(int a, int b) { x = a; y = b; }
    Punto add(Punto p) {
        x+=p.x; y+=p.y; return (*this);
    }
    Punto add(Punto p, Punto q) {
        x=x+p.x+q.x;
        y+=p.y+q.y;
        return (*this);
    }
    void inc23() { x+=2; y+=3; }
    void inc23(Punto p) { x+=p.x; y+=p.y; }
    void ver() const { cout << x << ", " << y << endl; }

    Punto operator+(Punto p) const; //añado un metodo a la clase Punto
    friend ostream& operator<<(ostream& s, Punto p); //añado una funcion amiga
};

Punto operator+(Punto p, int i); //funcion ajenas a la clase
Punto operator+(int i, Punto p); //funcion ajenas a la clase
ostream& operator<<(ostream& s, Punto p); //funcion ajenas a la clase (la meto como amiga)

//metodo
//se muestra 5 formas diferentes de implementarse
Punto Punto::operator+(Punto p) const {
    Punto paux; //constructor Punto()
    paux.add(Punto(x,y)); //metodo Punto Punto::add(Punto p)
    paux.add(p); //metodo Punto Punto::add(Punto p)
    return paux;

    Punto paux1; //constructor Punto()
    paux1.add(Punto(x,y), p); //metodo Punto Punto::add(Punto p, Punto q)
    return paux1;

    Punto paux2(x,y); //constructor Punto::Punto(int a, int b)
    paux2.add(p); //metodo Punto Punto::add(Punto p)
    return paux2;

    Punto paux3(x+p.x, y+p.y); //constructor Punto::Punto(int a, int b)
    return paux3;

    return Punto(x+p.x, y+p.y); //constructor Punto::Punto(int a, int b)
}

//funcion ajena a la clase Punto (solo puede invocar metodos publicos)
//se muestra 5 formas diferentes de implementarse
Punto operator+(Punto p, int i) {
    Punto paux(p); //constructor de copia
    paux.add(Punto(i)); //metodo Punto Punto::add(Punto p)
    return paux;

    Punto paux1(i); //constructor Punto::Punto(int a)
    paux1.add(p); //metodo Punto Punto::add(Punto p)
    return paux1;

    Punto paux2; //constructor Punto()//constructor de copia
    paux2.add(p, Punto(i)); //metodo Punto Punto::add(Punto p, Punto q)
    return paux2;

    Punto paux3, pi(i);
    paux3.operator+(p);
    paux3.operator+(pi);
    return paux3;

    Punto paux5(i);
    return p+paux5; //return p.operator+(paux5);
}

```

```

//funcion ajena a la clase Punto (solo puede invocar metodos publicos)
//se muestra 7 formas diferentes de implementarse
Punto operator+(int i, Punto p) {
    //idem funcion anterior (5 formas diferentes)

    return p+i;                //llamo a la funcion anterior

    return p.operator+(i); //lamo a la funcion anterior
}

//funcion amiga (si no seria imposible mostrar atributos x, y)
ostream& operator<<(ostream& s, Punto p) {
    s << "(" << p.x << ", " << p.y << ")";
    return s;
}

int main() {
    Punto x(3,2), y;
    const Punto z=1; //const Punto z(1); const Punto z=Punto(1);
    cout << "x, y, z valen:\n";
    x.ver(); y.ver(); z.ver();
    y.add(x, z);
    x=y.add(z);
    cout << "x, y, z valen:" << endl;
    x.ver(); y.ver(); z.ver();
    x.inc23(); y.inc23(z);
    cout << "x, y, z valen:" << endl;
    x.ver(); y.ver(); z.ver();

    //haz que sea posible ejecutar las lineas comentadas sin cambiar nada en la clase Punto
    //excepto añadir un método a la clase Punto y una funcion amiga
    x = y + z; cout << "x: "; x.ver();
    x = z + x; cout << "x: "; x.ver();
    y = z + 4; cout << "y: "; y.ver();
    y = 2 + z + y +1 ; cout << "y: "; y.ver();
    cout << "x: " << x << " y: " << y << " z: " << z << endl;

    system("PAUSE"); return EXIT_SUCCESS;
}

```

SALIDA POR PANTALLA

```

x, y, z valen:
3,2
0,0
1,0
x, y, z valen:
5,2
5,2
1,0
x, y, z valen:
7,5
6,2
1,0
x: 7,2
x: 8,2
y: 5,0
y: 9,0
x: (8, 2) y: (9, 0) z: (1, 0)
Presione una tecla para continuar . . .

```